

Build your DataOps support with CI/CD for Power BI and Fabric

Daniel PATKOS

Kevin CHANT



Power BI



Microsoft
Fabric



GitHub

Thank you for making this possible



28&29 May 2026
Clarion Congress Hotel Prague
CUBEX Centrum Prague



Speaker info - Kevin

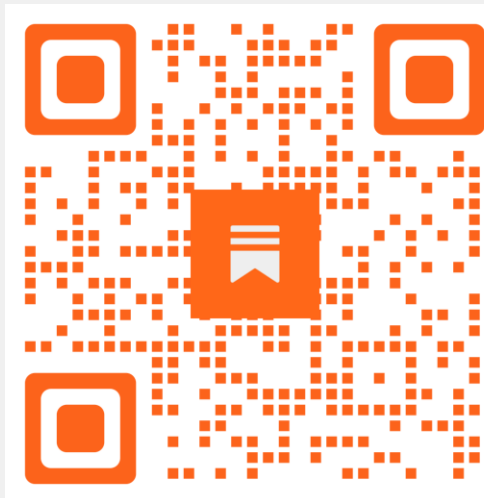
- Lead Technology Advocate in the Netherlands
- Experience in various sectors
- Various certifications, Data Platform MVP

- Twitter/Blue Sky: @kevchant
- LI: <https://www.linkedin.com/in/kevin-chant/>
- Blog: <https://www.KevinRChant.com>
- GitHub: <https://github.com/kevchant>



Speaker info

- Budapest, Hungary
- BI Architect @  VISUAL
LABS





Power BI & Git - Where Are You on the Journey?

1. I develop Power BI reports — not just view them.
2. I've struggled with version control (Project_Final_v7_REAL_FINAL.pbix.)
3. I've heard about the new Power BI Project file format (.pbip).
4. I know what TMDL is.
5. I have experience working with .pbip and TMDL.
6. I'm familiar with the meaning of basic git terminology – repo, branch, commit, merge, PR.
7. I use branches, pull requests, or code reviews as part of my workflow.

Recommended local applications

- Git (usually installed by other apps).
- Visual Studio Code, ideally with the below extensions:
 - Ideally with GitHub Copilot account setup
 - Mssql if working with SQL-related items
 - Fabric Studio
 - GitGraph
- Power BI Desktop when working with Power BI reports
- SQL Server Management Studio if working with only SQL items.
- Python for testing libraries locally (up to 3.13 if testing fabric-cicd)
- GitHub Desktop if require “Cherry picking”

Exercises

All of today's exercises can be downloaded from:

- <https://github.com/DataOpsSupport/DataPointExercises>

Accompanying repos inc. slides can be found at:

- <https://github.com/DataOpsSupport>

Scan to refresh your Git, CI/CD, and PBI knowledge



Agenda

1. 🦖 Dark Age – Before version control
2. 🌱 Local version control (benefits of .PBIP)
3. 🌿 GitHub & Power BI
Break
4. 🌳 GitHub Actions
🍴 Lunch break
5. 🌳 Microsoft Fabric Git integration
6. 🌳 Fabric CI/CD options
7. 🌳 Automated testing

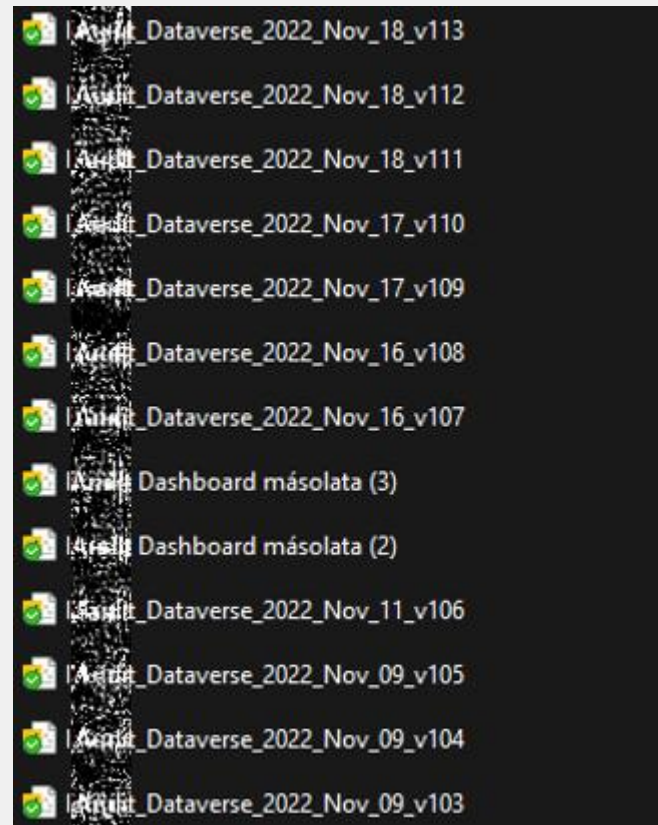


Dark Age

Before version control

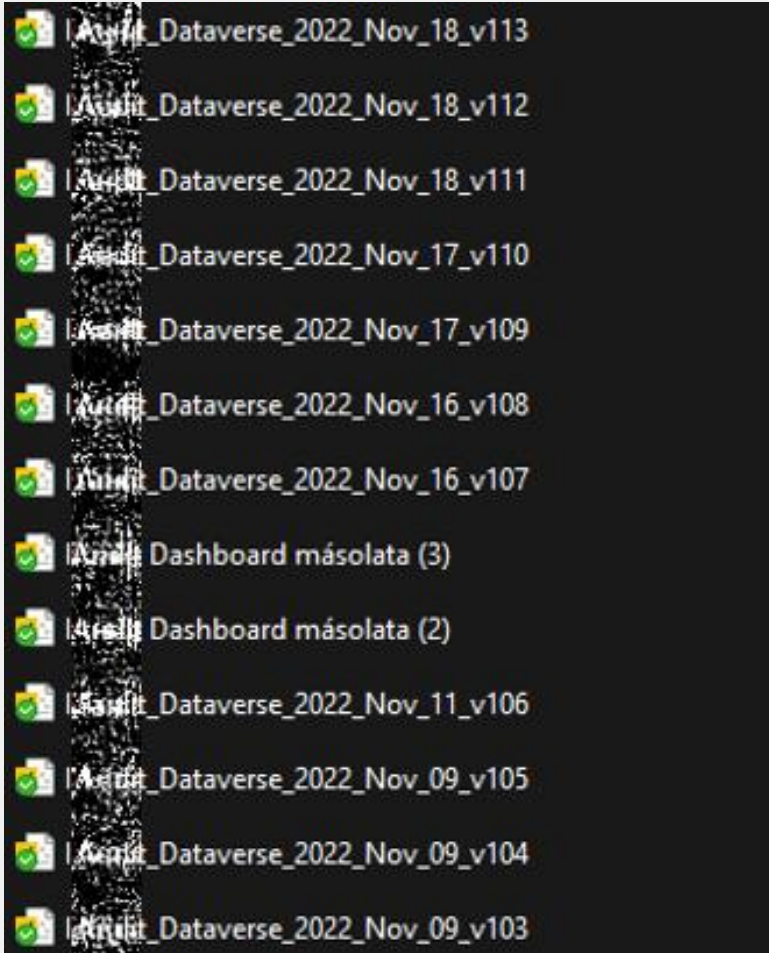
The Dark Age of Power BI Version Control

Developing on your own

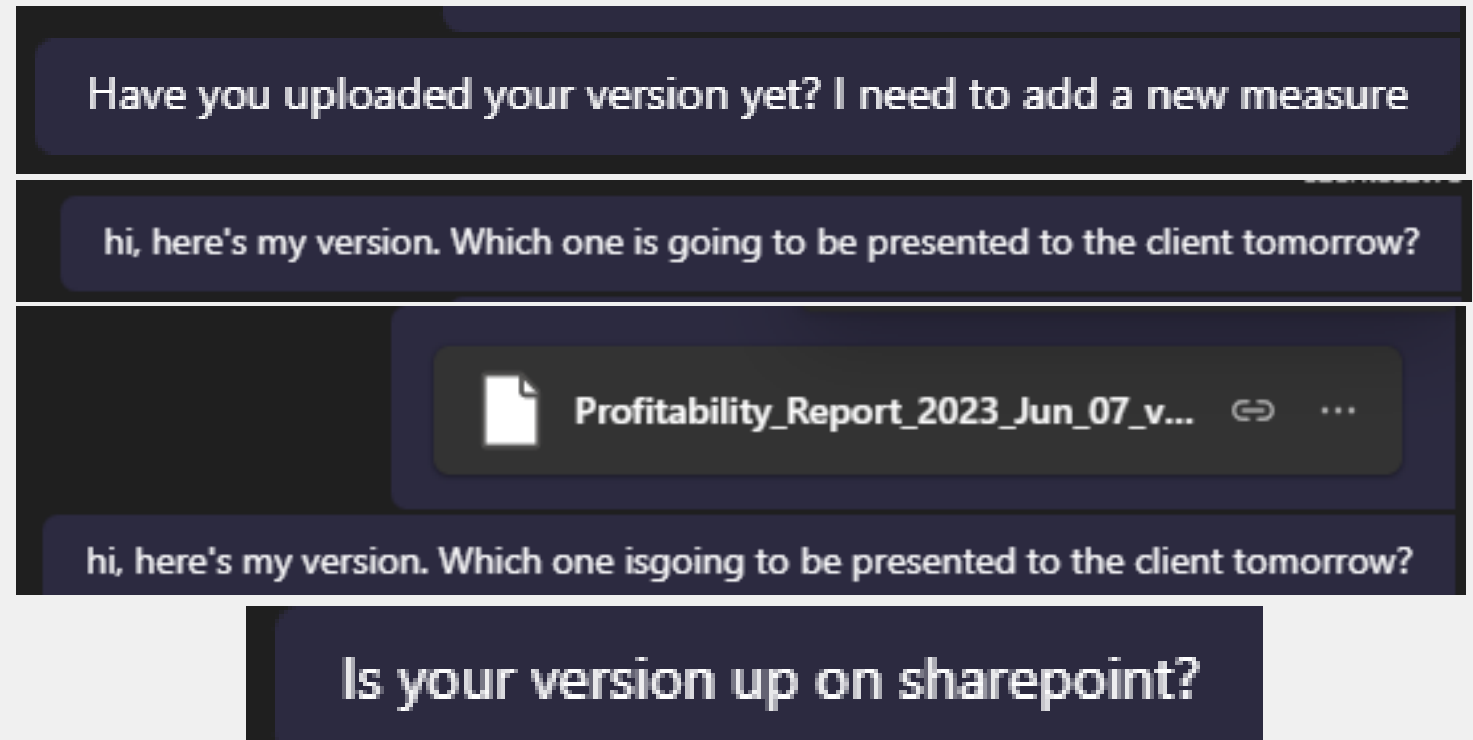


The Dark Age of Power BI Version Control

Developing on your own



Developing in teams



Workarounds & alternative solutions (🦖 Dark Age)

1) OneDrive

2) Rest API or Power Automate (Rest API)

3) External tools (e.g.: ALM Toolkit, pbi-tools, PBI Documenter) or Azure DevOps PBI Extension

4) Deployment pipelines (PPU or Premium)

👑 Power BI Project Files

- From .pbix to .pbib and .pbir (new file formats)

☐ Name	Date modified
📁 AdventureWorks.Report	19/03/2024 20:43
📁 AdventureWorks.SemanticModel	19/03/2024 20:43
⚙️ .gitignore	15/03/2024 10:43
📄 AdventureWorks.pbip	18/03/2024 12:21

- Option to switch to TMDL format



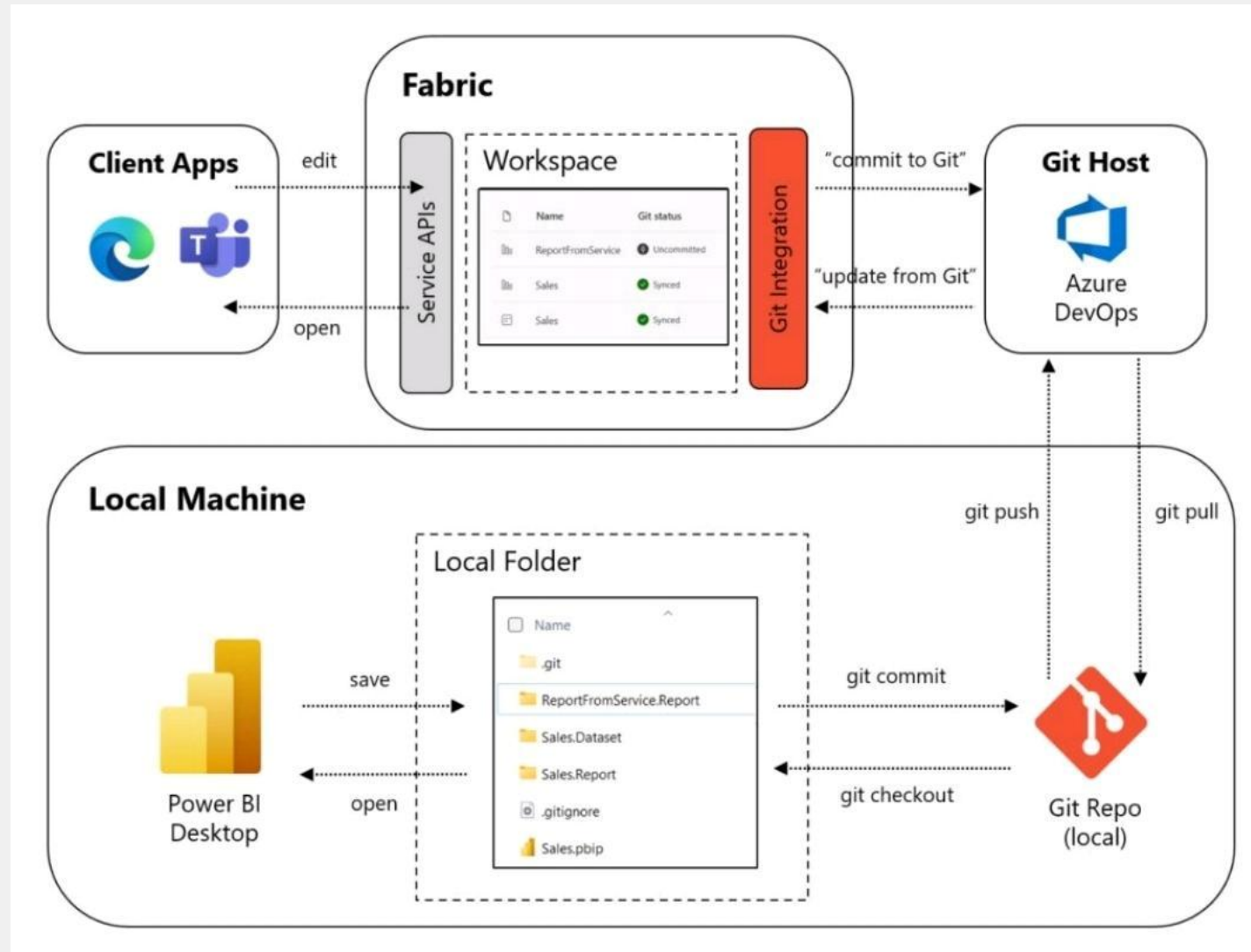
TMDL format

- Human friendly way to store semantic model related data
- Modular, granular editing: partial edits or deployments
- Uses a YAML-like syntax for easier reading
- Better integration with CI/CD and smoother version control
- Easy to parse (manually or with 3rd party tools)
- Option to bulk edit, create, remove – in code editors, Python or the new TMDL View

PBIR

- New(ish) format for report metadata.
- Uses a JSON syntax.
- Breaks down report items into a separate files.
- Integrates better with version control.
- Allows quick external editing in other tools.

How does it work?

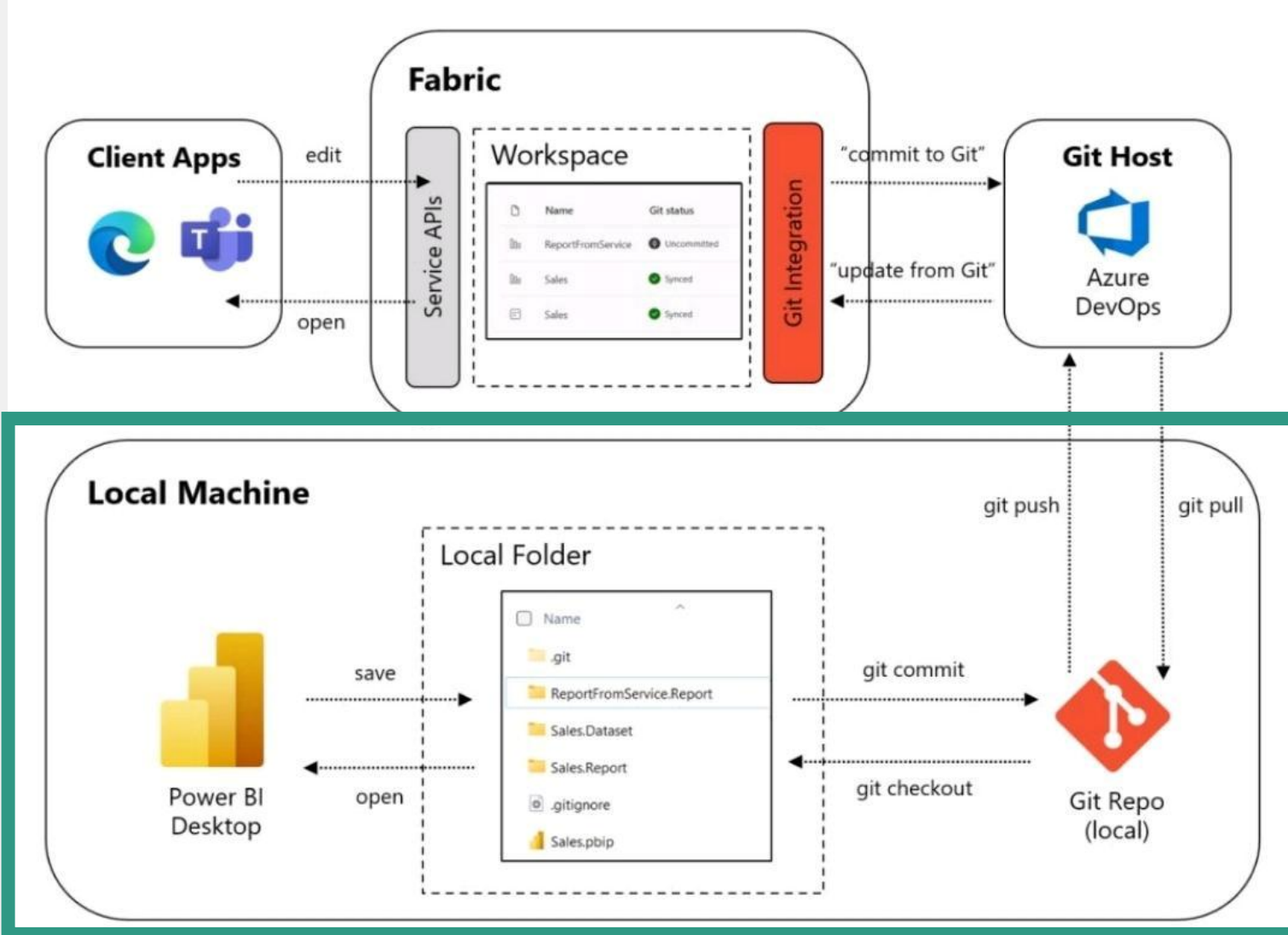




Local version control

Detailed, step-by-step guide

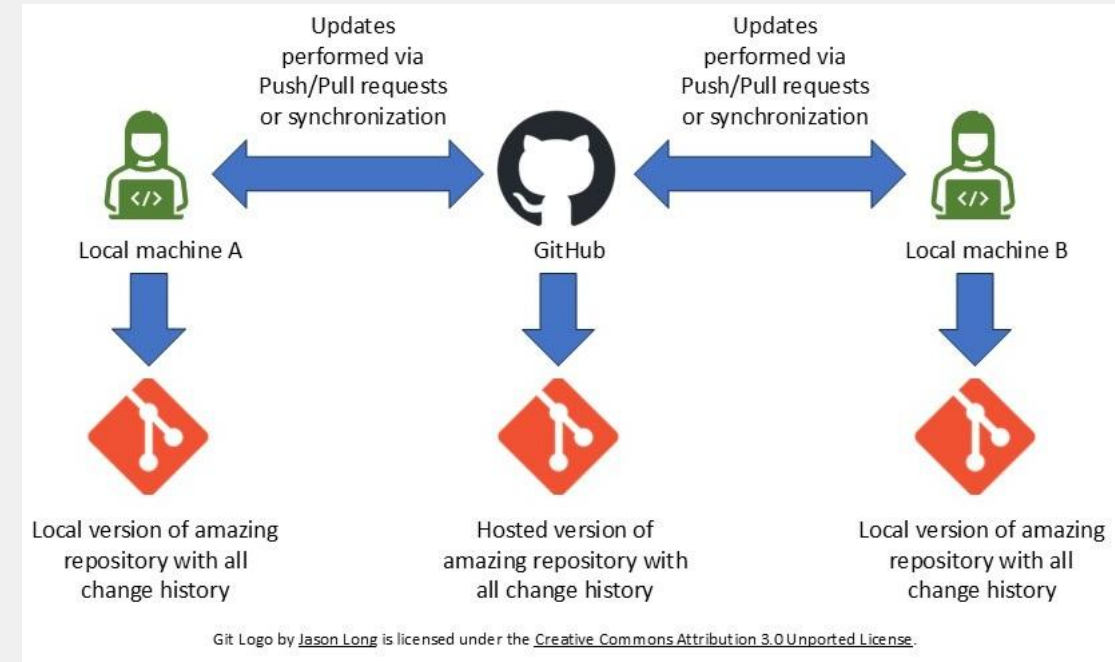
Version control in action....



Let's practice this part!

About Git

- 1) Distributed version control system
- 2) Built to handle updates to millions of lines of code
- 3) Operates via “hidden” database in folder
- 4) Open source, multi-platform



Git basics

- 1) **Repo[sitory]** – stores your project related files and track all changes made to files
- 2) **Commit** – snapshot of the current status of your project
- 3) **Branch** – separate workspace where you can develop and try new things w/o messing up the main project
- 4) **Merge** – integrate two different versions into one
- 5) **Pull Request (PR)** – formal way to ask for approval and merge changes of 2 branches
- 6) **Push** – Push your commits to a branch in remote repo
- 7) **Pull** – Get latest updates from a specific branch in remote repo



Exercise 1: Save a .pbix report as .pbip

Exercise 0: Set up git in VSCode

- One VSCode
- Open Terminal
- Run these commands:
 - `git config --global user.name "Your Name,,`
 - `git config --global user.email "your.email@example.com"`

Create local repo

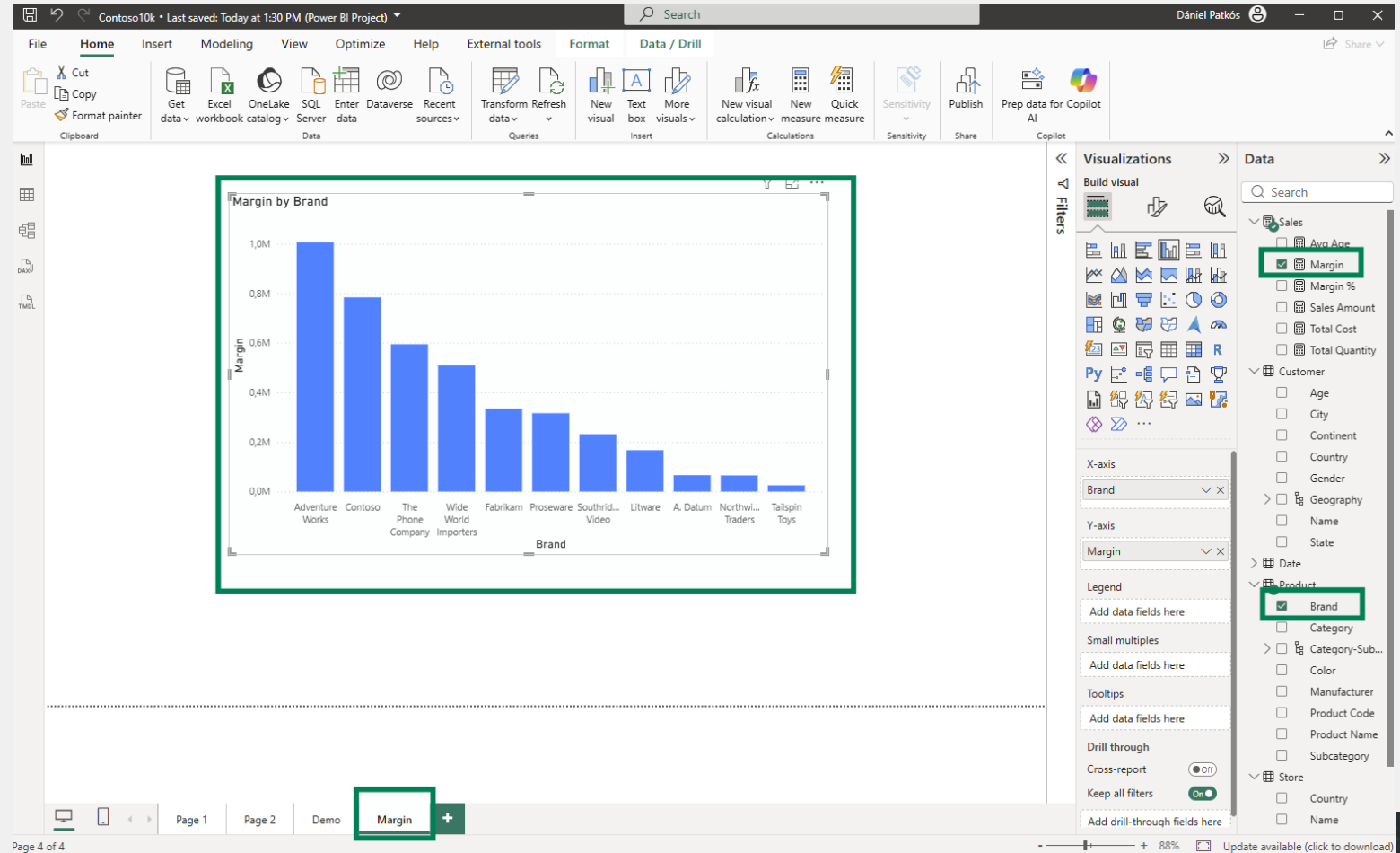
Create a new measure

Avg Age = AVERAGE (Customer[Age])

- Save
- Check changes
- Commit changes

Create a new page + visual

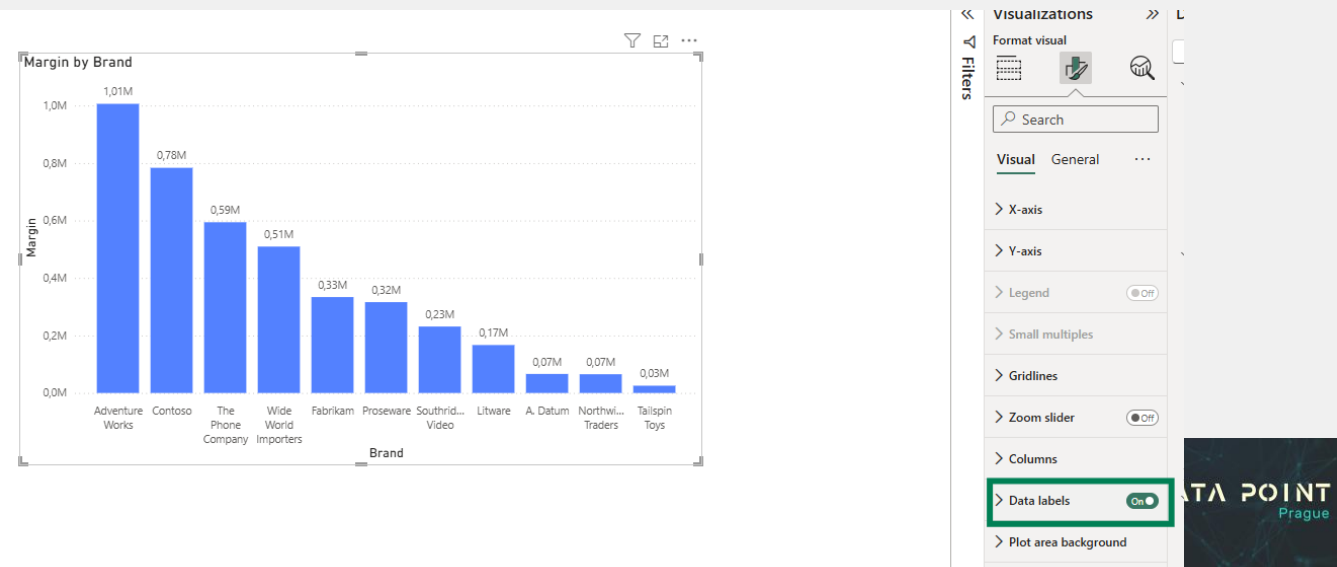
- Add new page
- Name: Margin
- New visual
- Brand by margin
- Save
- Check changes
- Commit changes



Make multiple changes 1/2

- Create new measure:
 - Double margin = [Margin] * 2
- Add background color to table on Demo page
- Add data labels to brand by margin visual
- Save
- Check changes
- DO NOT stage Double Margin
- Commit only visualization related
- Discard Double margin

Brand	Sales Amount
A. Datum	113 310,45
Adventure Works	1 772 421,61
Contoso	1 390 583,44
Fabrikam	591 240,26
Litware	290 189,30
Northwind Traders	116 691,16
Proseware	550 225,91
Southridge Video	411 308,07
Tailspin Toys	49 322,27
The Phone Company	1 103 284,41
Wide World Importers	917 362,06
Total	7 305 938,94



Make multiple changes 2/2

- DO NOT stage Double Margin
- Commit only visualization related changes
- Discard Double margin
- Check Power BI
- Close Power BI Desktop
- Reopen Power BI Desktop
- Look for Double Margin

The screenshot displays the Power BI Desktop interface. The 'SOURCE CONTROL' pane on the right shows a 'CHANGES' section with a 'Commit' button. Below it, a list of changes is visible, including 'Sales.tmdl' and 'Contoso10k.SemanticModel\defi...'. A green box highlights the 'Commit' button. The main area shows a table of sales data with columns 'Brand' and 'Sales Amount'. The table includes rows for various brands like 'A. Datum', 'Adventure Works', 'Contoso', 'Fabrikam', 'Litware', 'Northwind Traders', 'Proseware', 'Southridge Video', 'Tailspin Toys', 'The Phone Company', and 'Wide World Importers', along with a 'Total' row. A green box highlights the 'Total' row. The 'Visualizations' pane on the right shows a list of fields including 'Sales', 'Avg Age', 'Margin', 'Margin %', 'Sales Amount', 'Total Cost', and 'Total Quantity'. A green box highlights the 'Sales' field.

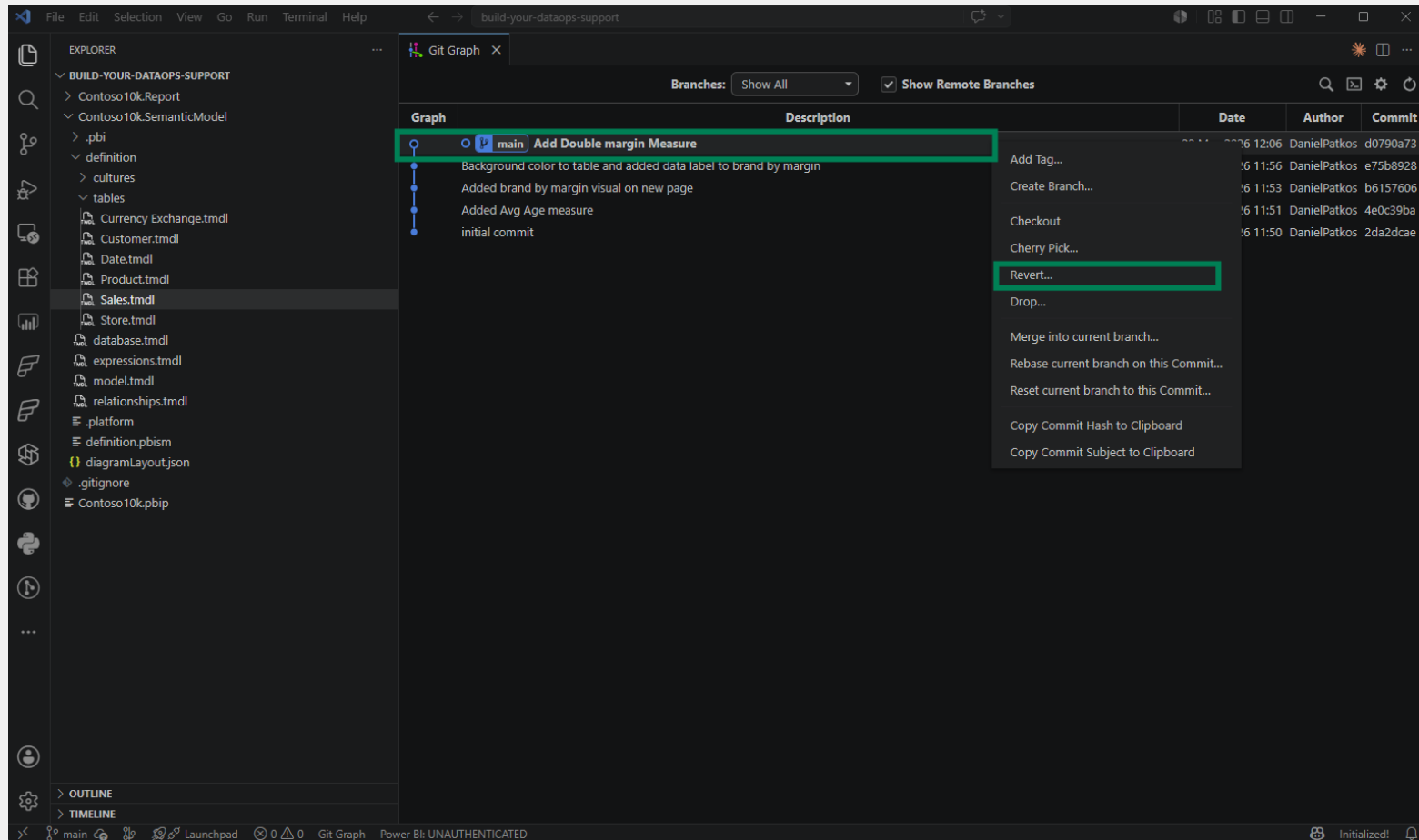
Brand	Sales Amount
A. Datum	113 310,45
Adventure Works	1 772 421,61
Contoso	1 390 583,44
Fabrikam	591 240,26
Litware	290 189,30
Northwind Traders	116 691,16
Proseware	550 225,91
Southridge Video	411 308,07
Tailspin Toys	49 322,27
The Phone Company	1 103 284,41
Wide World Importers	917 362,06
Total	7 305 938,94

Add a new measure and commit

- Create new measure:
 - Double margin = $[\text{Margin}] * 2$
- Save
- Check changes in VS Code
- Commit

Revert to previous commit

- Revert in VS Code
- Check PBI
- Close PBI
- Reopen PBI
- Look for Double Margin



Create a new branch

- feature/pie_chart
- Always separate your feature developemnt and your bugfixes from the main branch
- Use feature branches
- This makes collaboration easier



GitHub & Power BI Service

Share with the team and automate the deployment

GitHub

- Manages Application Lifecycle Management
- Popular with open-source community
- Various plans
- Large range of features

GitHub Plans (formerly products)

- GitHub Free for personal accounts
- GitHub Pro
- GitHub Free for organizations
- GitHub Team
- GitHub Enterprise
 - Cloud
 - Server

As well as repositories

- GitHub Actions
- GitHub Copilot
- GitHub issues
- Wikis

General security considerations

- Use secrets and Azure Key Vault with GitHub Actions.
- Enable code and secret scanning.
- GitHub has article on what to do if sensitive values appear in repository.
- Recommend GitHub Enterprise for production use.
- Consider GitHub certification training.

GitHub certifications available

- GitHub Foundation
- GitHub Administration
- GitHub Actions
- GitHub Security
- GitHub Copilot

Recommended GitHub resources

- GitHub documentation
- Microsoft Learn

Tip: For GH-300 exam recommend the online series by Peter de Tender

GitHub and associated apps walkthrough



Exercise 2: PUSH the current repo to GitHub

Make changes, create PR

Cherrypick

- Rebase
- Merge solution

Let's solve a merge conflict 1/2

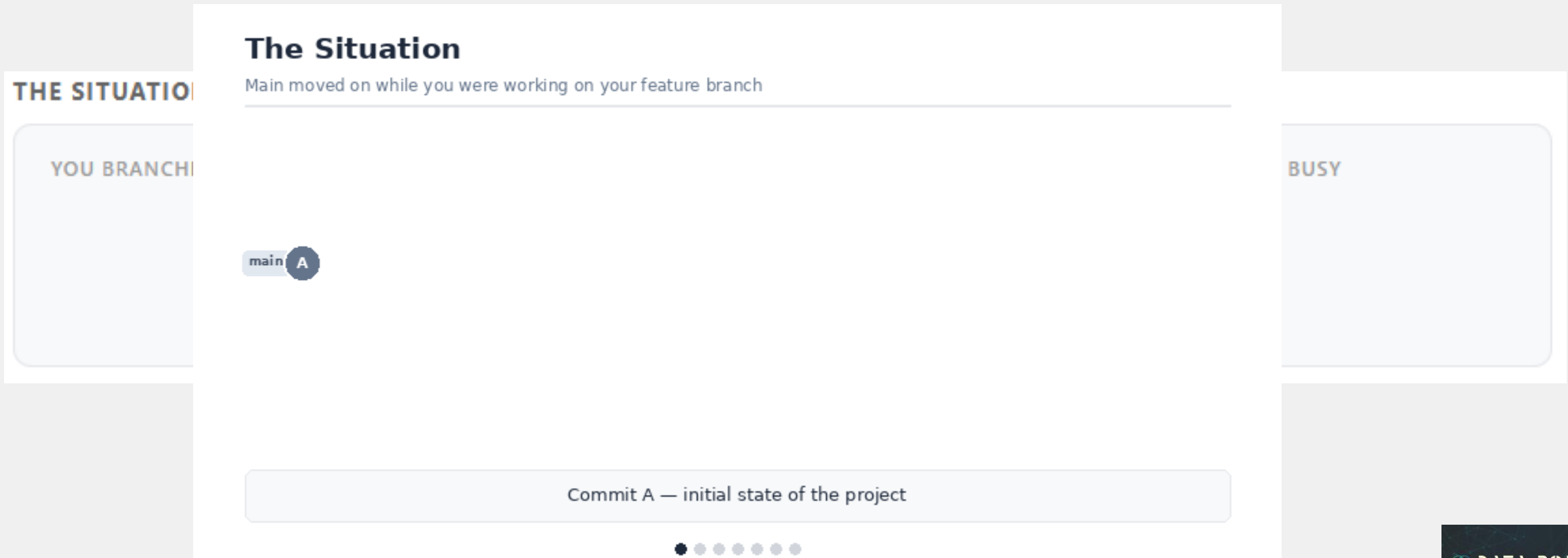
- Create a new branch from main
 - Open the report
 - Click to the first page
 - Save
 - Commit
 - Switch to main
 - Merge this branch into main
- Create a new branch from main
 - Open the report
 - Click to the last page
 - Save
 - Commit

Let's solve a merge conflict 2/2

- Switch to main try to merge 2nd branch into it
- Check error message
- Solve conflict
 - Options:
 - Accept current
 - Accept incoming
 - Manual
- Commit
- Check result in PBI

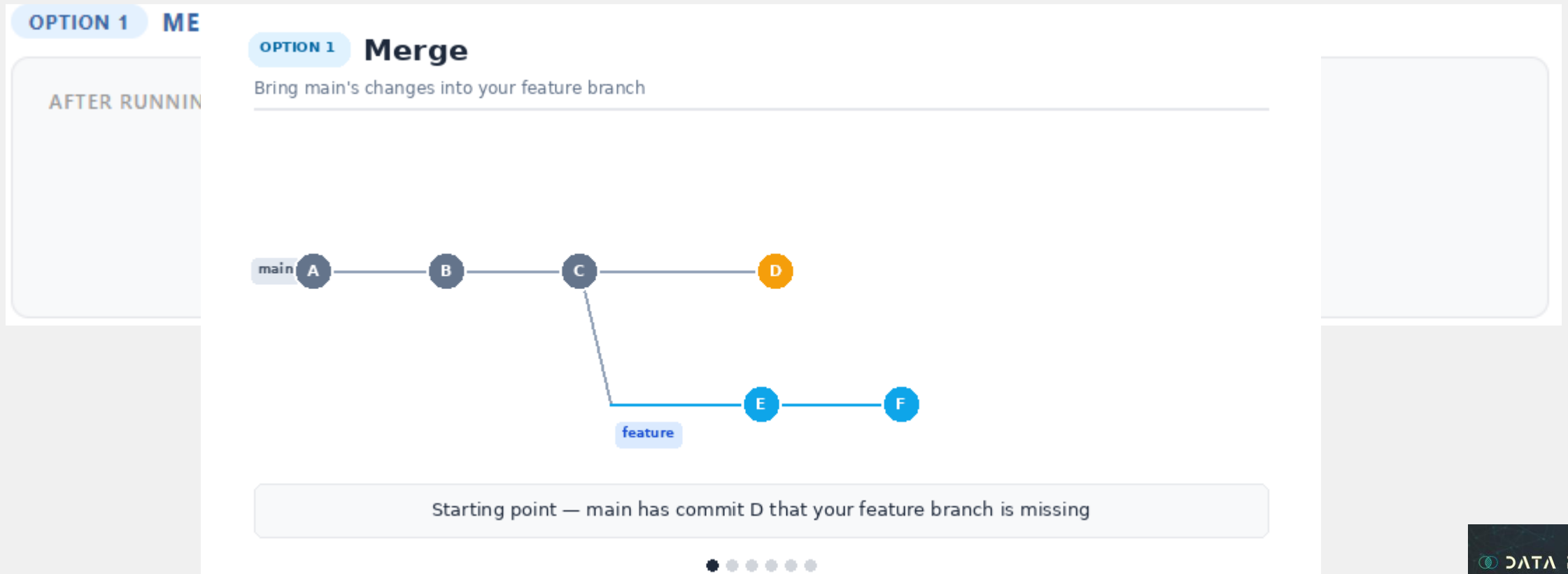
How to stay in sync when others are also working on the project

Main moved on. Now what?



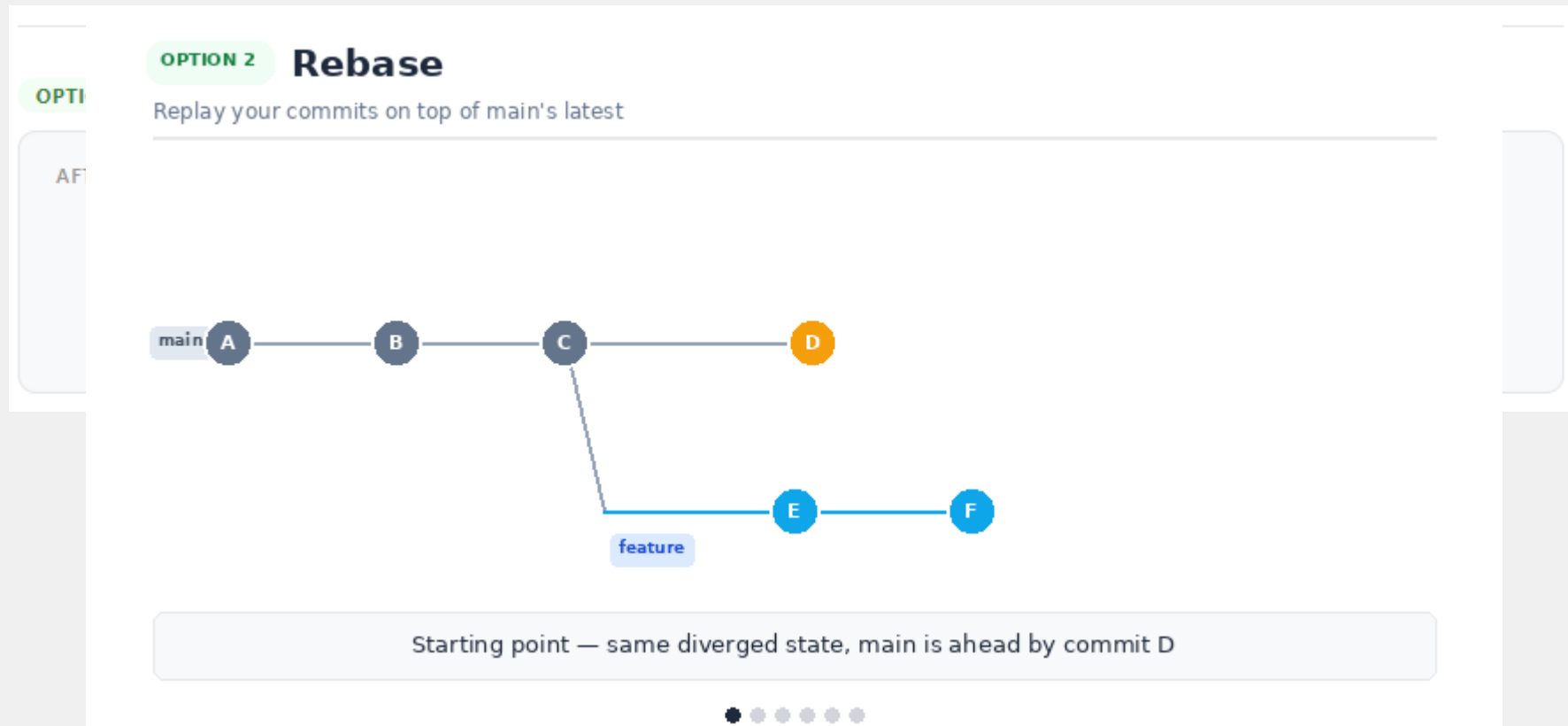
How to stay in sync when others are also working on the project

Main moved on. Now what?



How to stay in sync when others are also working on the project

Main moved on. Now what?



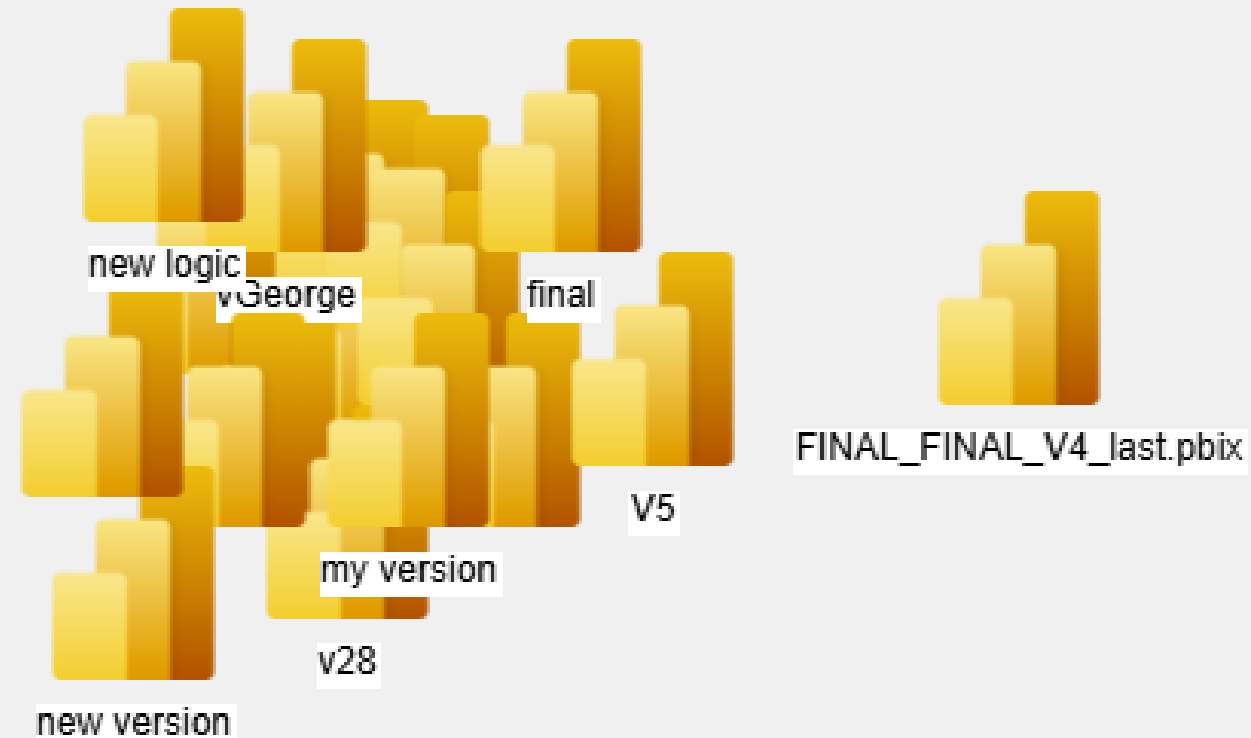


Here comes the cavalry

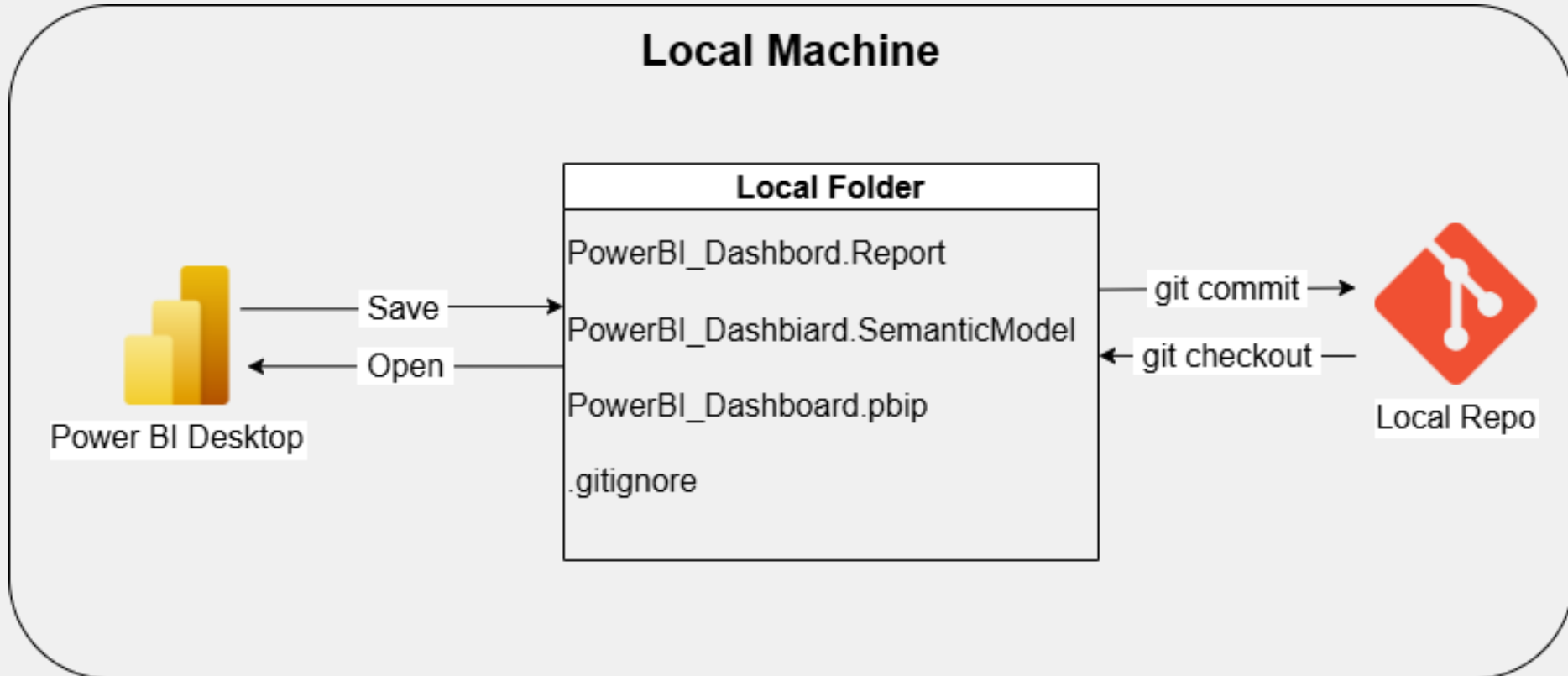
GitHub Actions terminology

- **Workflow** – Configurable process to run jobs (pipelines)
- **Action** – Reusable jobs or code.
- **Variables** – predefined values (configured either in or out of workflow)
- **Parameters** – Values entered when workflow started.
- **Secrets** – Encrypted values stored in GitHub.

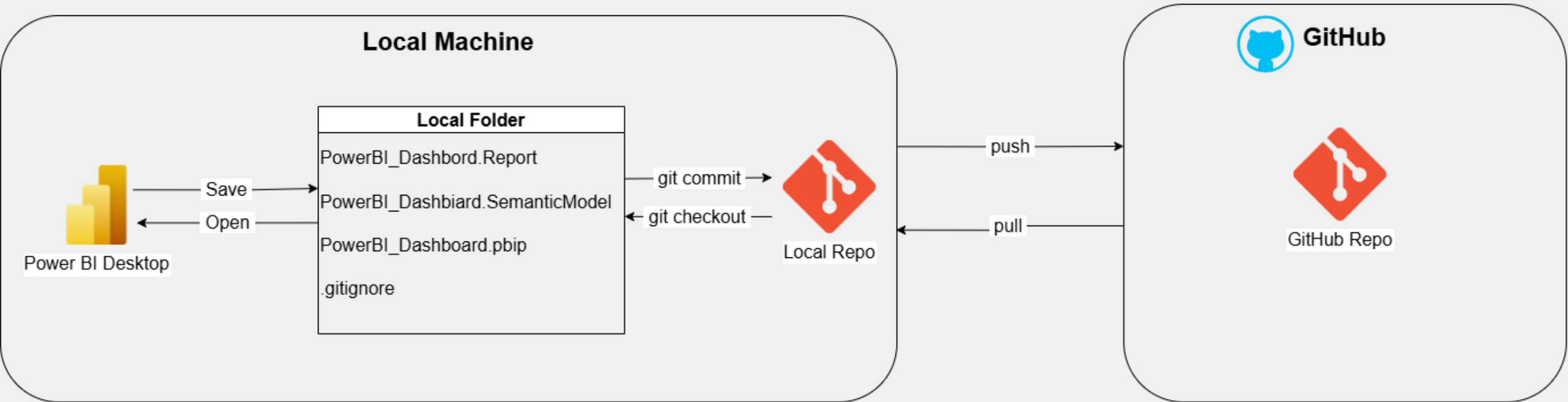
Version Chaos



Local version control



Repo stored in GitHub



Your next challenge: How do you...

1. Ensure consistency across multiple Power BI projects?
2. Stop suboptimal practices from leaking into production?
3. Align developers with different habits and skill levels?
4. Automate security and quality checks reliably?

Beyond version control: PBI meets CI/CD

- Apply security measure on your main branch(es)
 - Run automated tests
 - *Automate security checks and approvals*
-
- Use familiar 3rd party tools like: Tabular Editor and run:
 - Best Practice Analyzer
 - Format DAX
 - Any macros

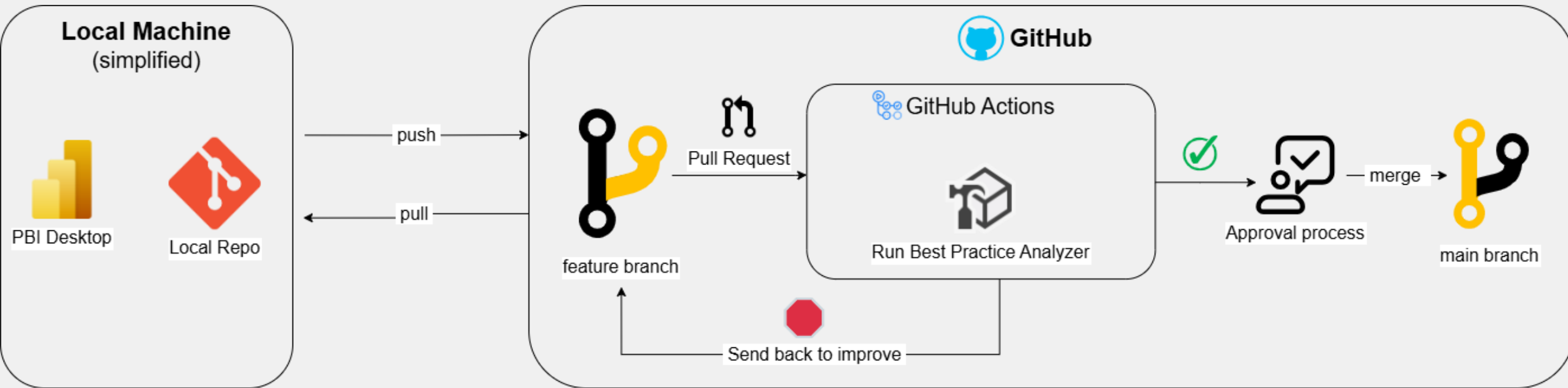


Beyond version control: PBI meets CI/CD

Best Practice Analyzer	
Show ignored	
Object	Type
+ Don't summarize numeric columns (22 objects)	
- Avoid division (use DIVIDE function instead) (12 objects)	
[Internet Current Quarter Gross Profit Performa...	Measure
[Internet Current Quarter Sales Performance]	Measure
[Internet Previous Quarter Gross Profit Proporti...	Measure
[Internet Previous Quarter Sales Proportion to Q...	Measure
[Reseller Current Quarter Sales Performance]	Measure
[Reseller Previous Quarter Gross Profit Proportio...	Measure
[Reseller Previous Quarter Sales Proportion to Q...	Measure
[Reseller Current Quarter Gross Profit Performa...	Measure
[Total Current Quarter Gross Profit Performance]	Measure
[Total Current Quarter Sales Performance]	Measure
[Total Previous Quarter Gross Profit Proportion t...	Measure
[Total Previous Quarter Sales Proportion to QTD]	Measure
- Do not use floating point data types (4 objects)	
'Reseller Sales'[Discount Amount]	Column
'Reseller Sales'[Unit Price Discount Pct]	Column
'CurrencyRate'[AverageRate]	Column
'CurrencyRate'[EndOfDayRate]	Column
+ Avoid empty descriptions (62 objects)	
+ Add objects to perspectives (30 objects)	
+ Organize columns and hierarchies in display folders (7 objects)	
+ Organize measures in display folders (3 objects)	
146 objects in violation of 8 Best Practice rules.	

```
1 [{
2   "Name": "Avoid division (use DIVIDE function instead)",
3   "Category": "DAX Expressions",
4   "CustomSeverity": 3,
5   "Description": "Calculated Columns, Measures or Calculated Tables should not use the division symbol (/) unless the denominator is a constant value. Instead, it is advised to always use the DIVIDE(<numerator>,<denominator>) function.",
6   "Severity": 3,
7   "Scope": "Measure, CalculatedColumn, CalculatedTable",
8   "Expression": "Tokenize().Any( Type = DIV and Next.Type <> INTEGER_LITERAL and Next.Type <> REAL_LITERAL )",
9   "CompatibilityLevel": 1200,
10  "ID_Original": "DAX_DIVISION_COLUMNS",
11  "ID": 52,
12  "Anchor": "avoid-division-use-divide-function-instead"
13 },
14 {
15   "Name": "[Performance] Check if relationship is many-to-many (N:N)",
16   "Category": "Performance",
17   "CustomSeverity": 3,
18   "Description": "Many-to-many (N:N) relationships can lead to performance issues or incorrect results if not implemented carefully. Evaluate the necessity of such relationships.",
19   "Severity": 3,
20   "Scope": "Relationship",
21   "Expression": "FromCardinality.ToString() = \"Many\" and ToCardinality.ToString() = \"Many\"",
22   "CompatibilityLevel": 1200,
23   "ID_Original": "CHECK_MANY_TO_MANY_RELATIONSHIPS",
24   "ID": 74,
25   "Anchor": "performance-check-if-relationship-is-many-to-many"
26 },
27 {
28   "Name": "[Performance] Check if relationship is bi-directional",
29   "Category": "Performance",
30   "CustomSeverity": 3,
31   "Description": "Bi-directional relationships can introduce ambiguity and performance issues. Use them only when necessary and well understood.",
32   "Severity": 3,
33   "Scope": "Relationship",
34   "Expression": "CrossFilteringBehavior == CrossFilteringBehavior.BothDirections",
35   "CompatibilityLevel": 1200,
36   "ID_Original": "CHECK_BI_DIRECTIONAL_RELATIONSHIPS",
37   "ID": 75,
38   "Anchor": "performance-check-if-relationship-is-bi-directional"
39 }
40 ]
```

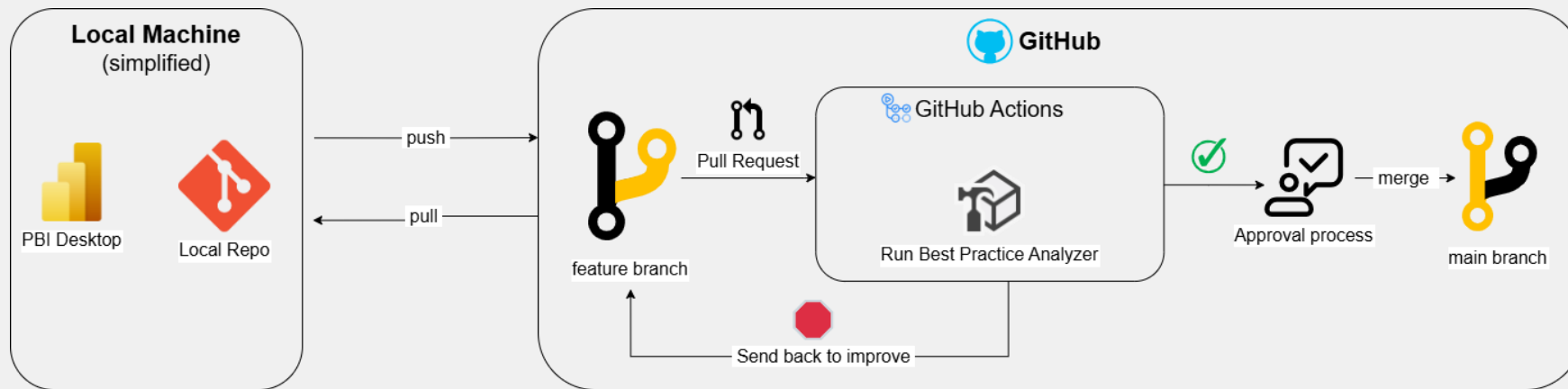
PR triggered BPA analysis



To summarize

- Power BI version control modernizes version control for reports.
- Traditional Git practices can now be performed for Power BI.
- GitHub can support your Power BI version control.

This is just the beginning!



Microsoft Fabric Git integration

About Microsoft Fabric Git Integration

- Native Git-based version control for Fabric items.
- Allows supported items in a workspace to have metadata synchronized with a Git repository.
 - To be more precise a workspace synchronizes with a branch.
- Supports cloud-based versions of Azure DevOps and GitHub.
- Requires Fabric or Power BI Premium capacity.
- Items supported at various levels.

Examples of Supported items

- **Data Agents (preview)**
- **ML items (preview)**
- Lakehouse
- Notebooks
- Spark Job Definitions
- Spark environment
- Dataflows gen2
- Mirrored database
- Warehouses (preview)
- SQL database
- Eventhouse and KQL database
- Activator (preview)
- Maps (preview)
- EventStream
- Reports (with exceptions)
- Semantic models (with exceptions) (preview)
- Org app (preview)
- Graph in Fabric (preview)

Benefits of Git integration

- Can track and manage changes to schemas of Fabric items.
- Reduces conflicts and allows history of changes.
- Can work in Fabric natively.
- API alternatives possible.
- Supports branch-based development.

Features include

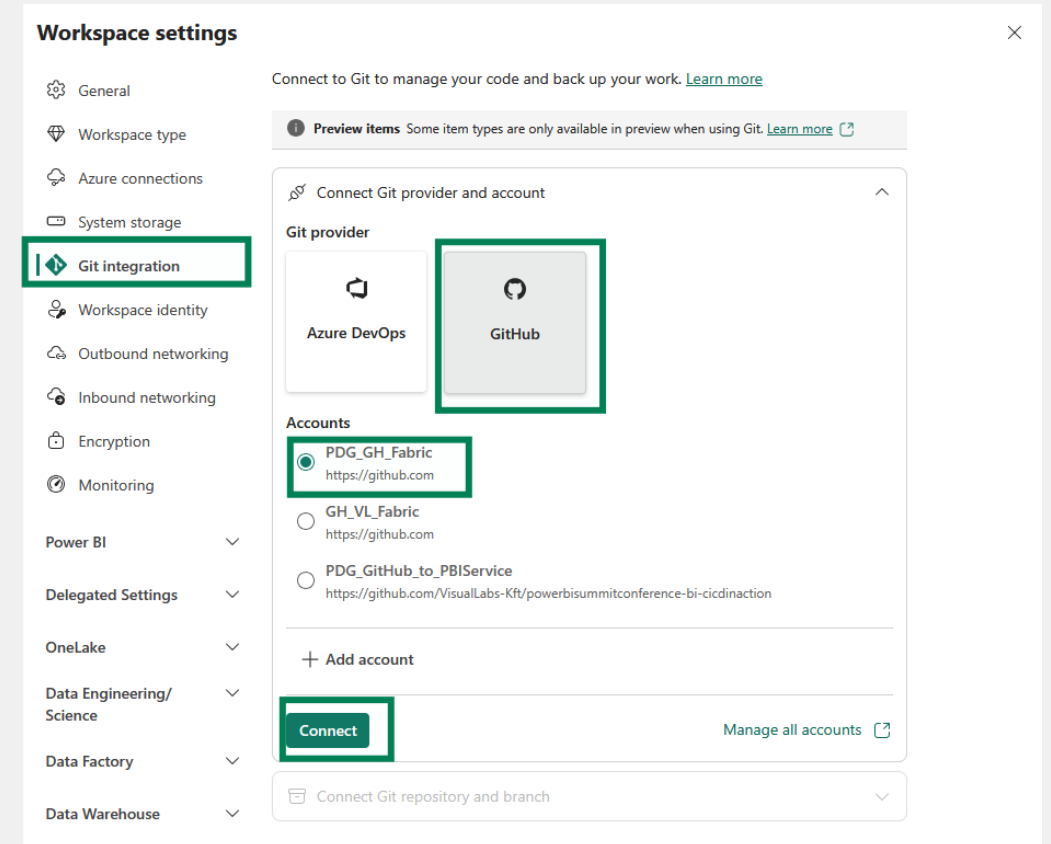
- Nice GUI manage & view changes to Fabric items.
- Branch out to new workspace/branch.
 - Either all items or ones you select.
- Introduces new deployment possibilities for Fabric items.

Configuring Microsoft Fabric Git Integration with GitHub

- Utilizes Personal Access Tokens.
- User requires access to both workspace and repository.
- Unsupported items can still be deployed in the workspace

Configure Microsoft Fabric Git integration with GitHub

- Create your Personal Access Token in GitHub
- Go to your Fabric Workspace
- Click Workspace settings



Branching out to new workspace

- Allows you to essentially copy a workspace and/or branch.
- Typically copy a development workspace & branch to feature workspace.
- Schema of items copied into new workspace & branch.
- Ideal for recommended development process.

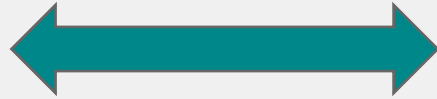
Current ways to branch out to new workspace

1. With GUI to a new workspace & workspace.
2. With GUI to existing workspace & new branch.
3. Automate with either Azure DevOps or GitHub.

Git integration Demo

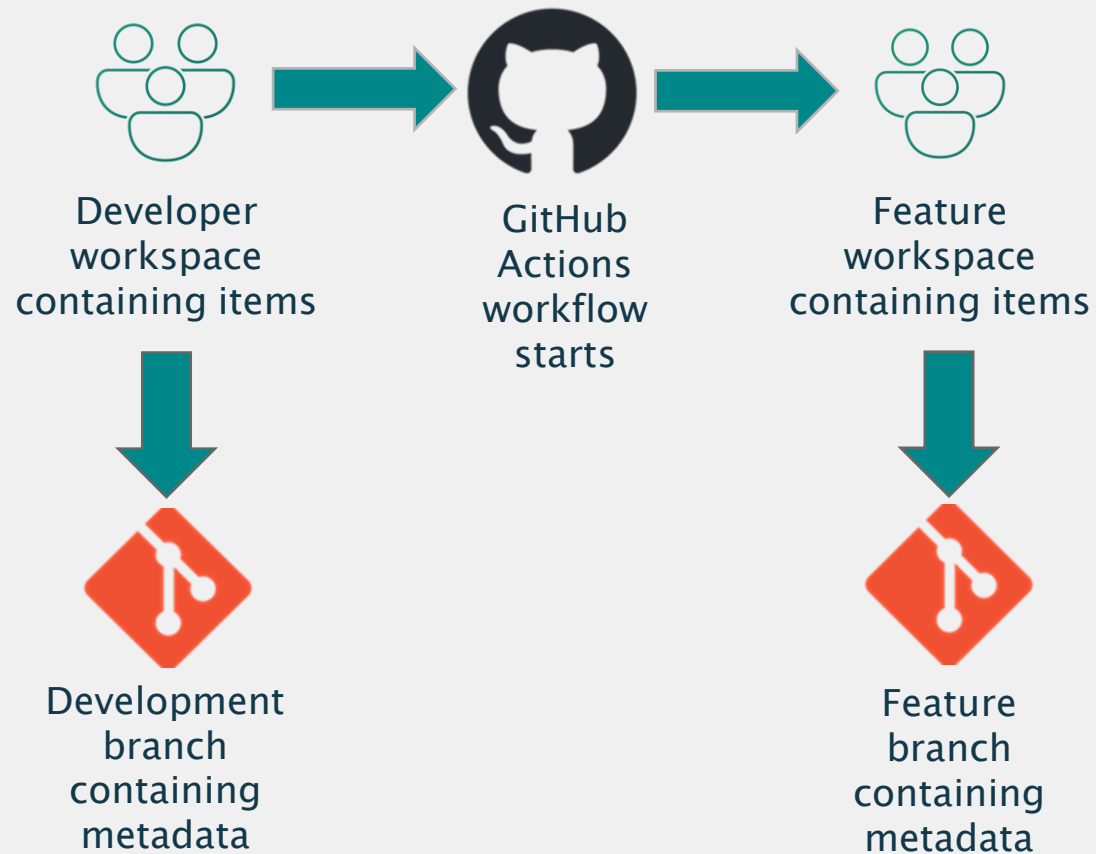


Microsoft Fabric workspace



GitHub

Automated branching out demo



Git Logo by [Jason Long](#) is licensed under the [Creative Commons Attribution 3.0 Unported License](#).

Fabric Exercise 1: Configure Git integration

Security considerations

- Where possible, Keep organization in relevant region.
- Use GitHub policies where applicable.
- Avoid exposing sensitive details in Repos & Workflows.
- Consider GitHub Advanced Security features.



Keep in mind

- Direct edits on PBI Service are displayed immediately – no need for approvals, you have the chance to push the changes back to GH
- You can still publish and overwrite your file on PBI Service from PBI Desktop
- On GH side you won't get any notification about changes made on PBI Service – if not committed

Fabric CI/CD options

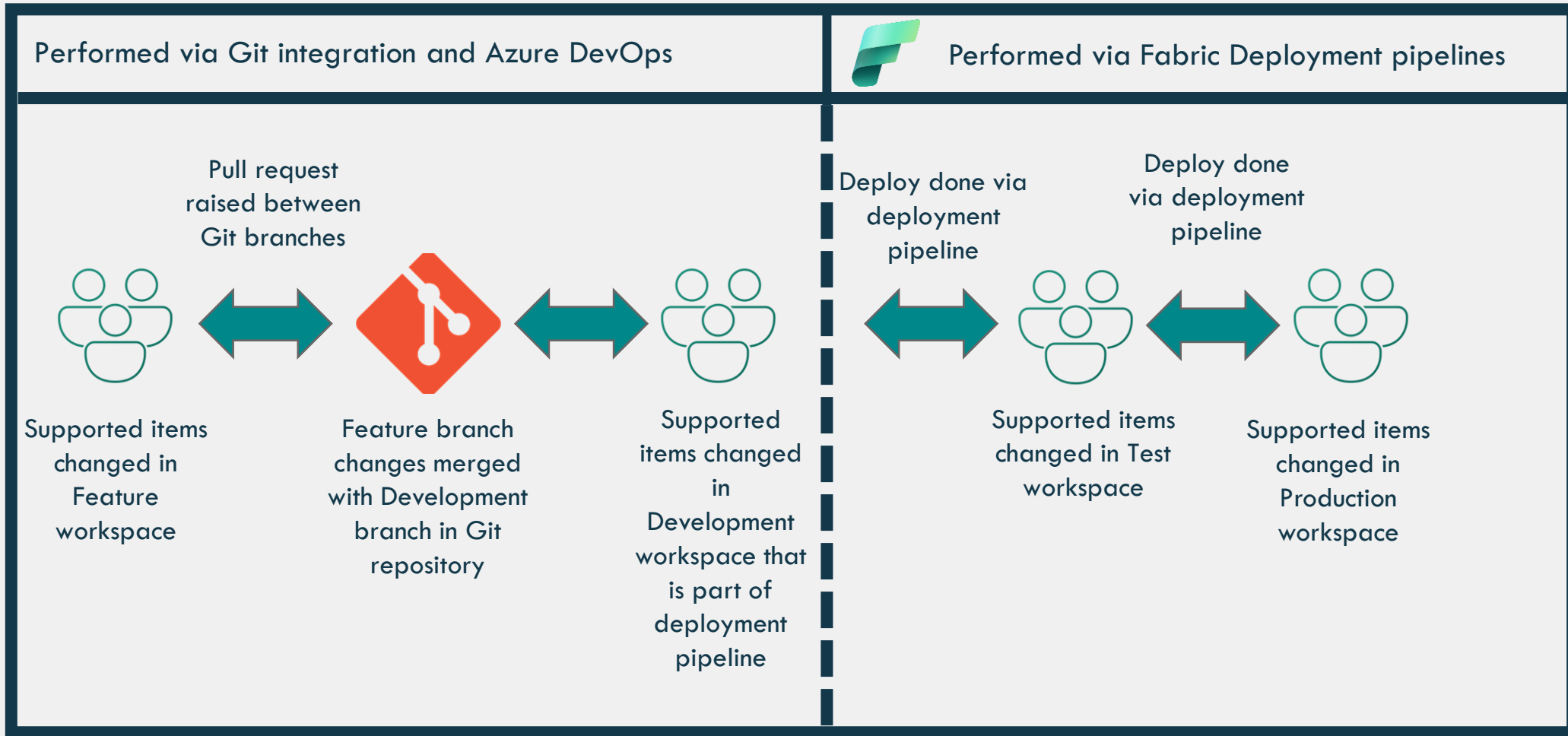
Two main ways to perform CD when using GitHub for CI

- Natively within Microsoft Fabric.
- Orchestrate with GitHub

Microsoft Fabric Deployment Pipelines

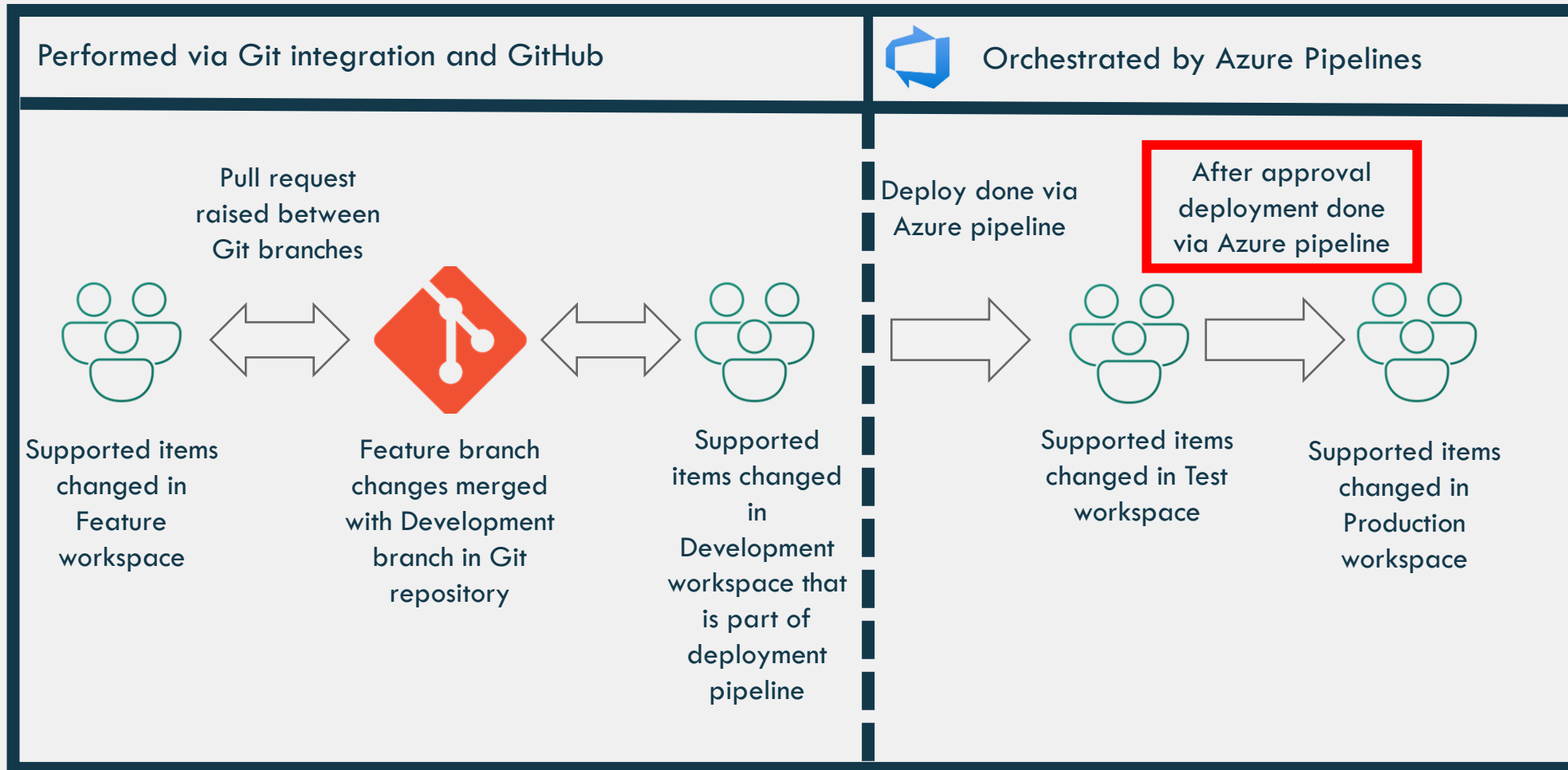
- Low code/no-code solution native in Fabric
- Requires Power BI Premium or fabric capacity
- Ideal for business citizens & low complexity requirements

Microsoft Fabric Deployment Pipelines



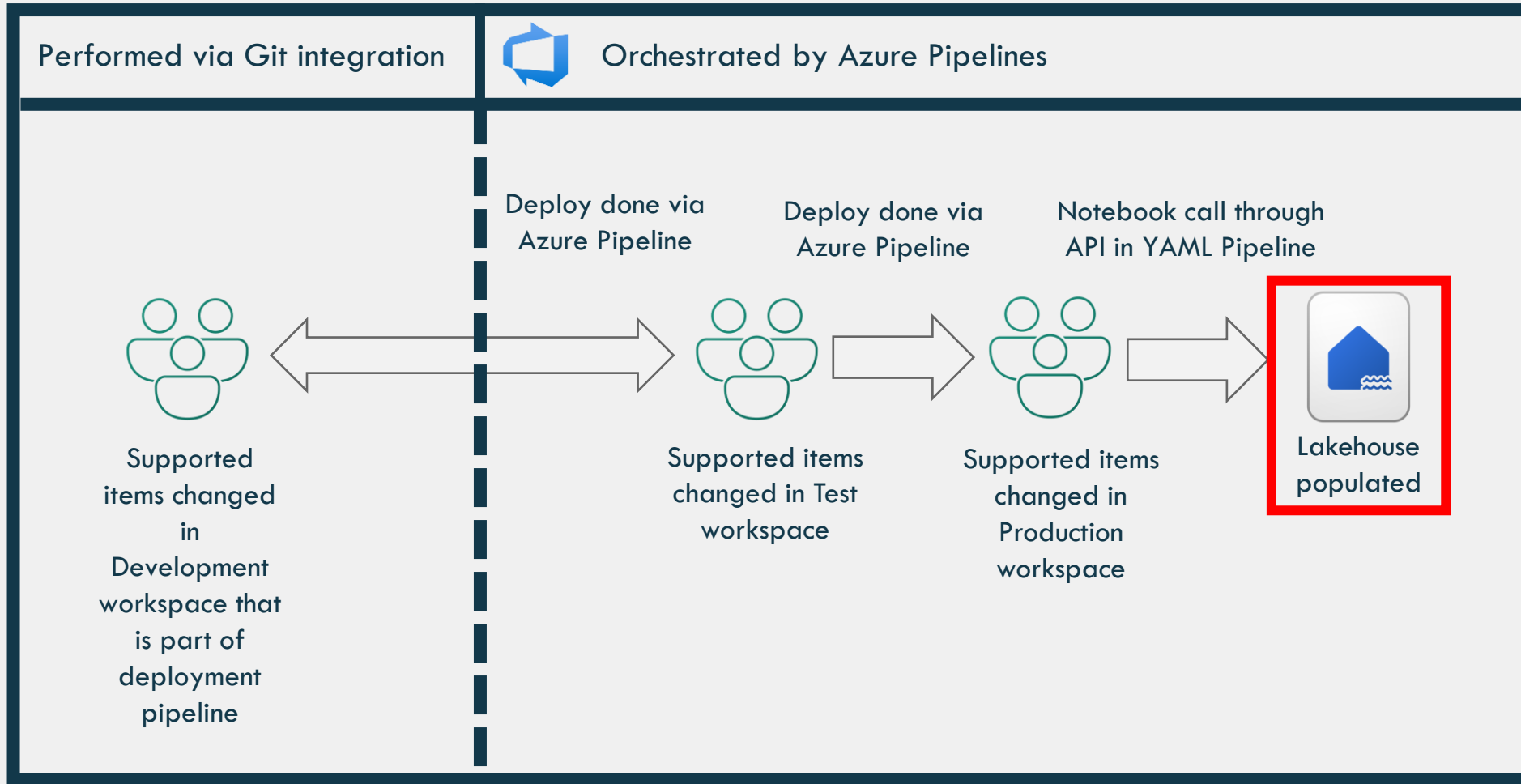
Git Logo by [Jason Long](#) is licensed under the [Creative Commons Attribution 3.0 Unported License](#).

Orchestrated by Azure Pipelines



Git Logo by [Jason Long](#) is licensed under the [Creative Commons Attribution 3.0 Unported License](#).

Another advantage of Azure Pipelines



Demo

- Orchestrating Deployment Pipeline in GitHub Actions workflow

Two main ways to perform CD with GitHub Actions workflows

- Merging Git branches
- Git-based deployments using GitHub Actions workflows



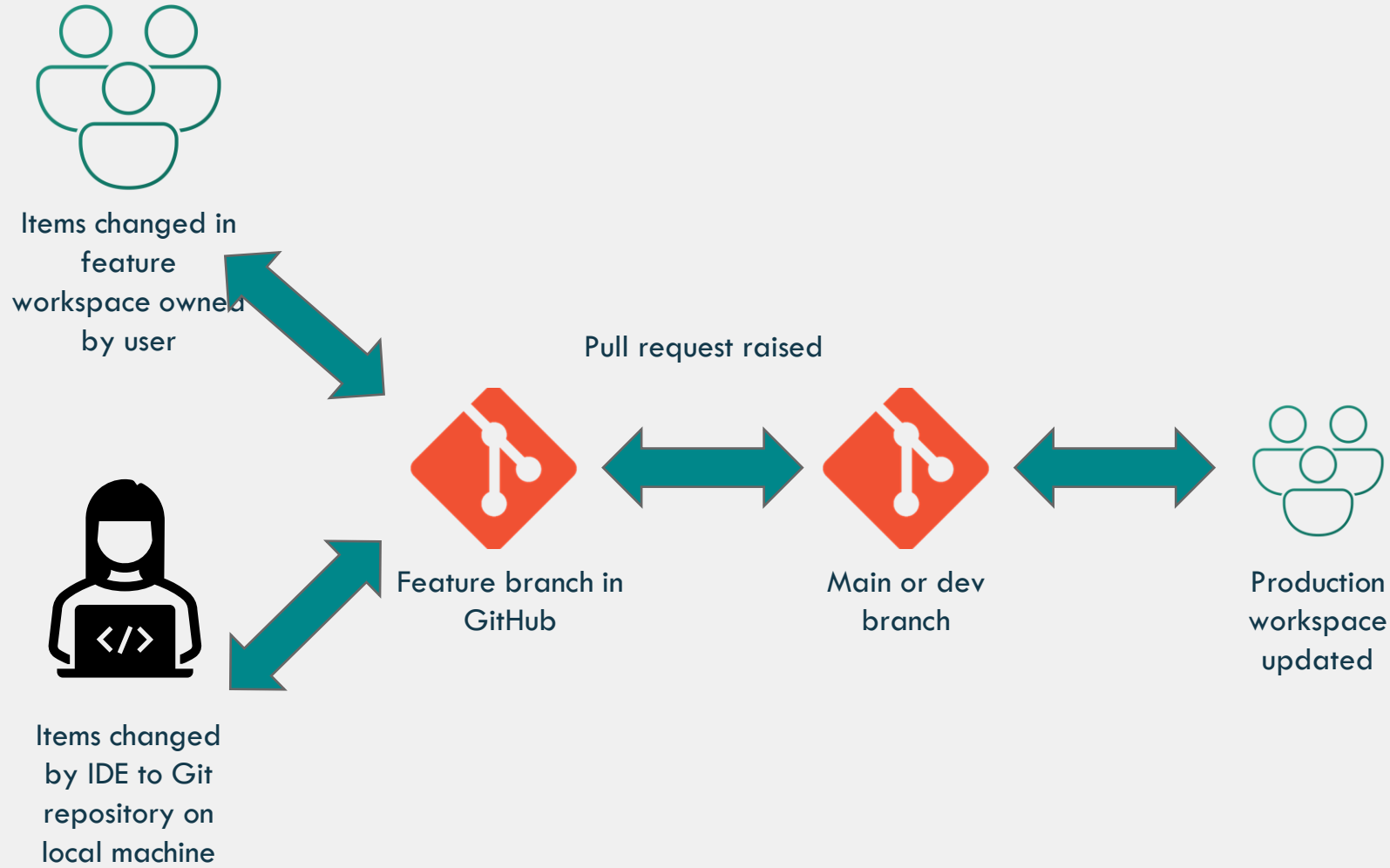
Merging Git branches

- Configure Git integration on multiple workspaces connected to same Git repository.
- Merge updates between the various branches.
- Can be done manually or with API's.
- Additional code can be done to perform tasks like rebindings.
- Ideal when want version control for all workspaces.

Git API examples

- Connect and disconnect workspace from branch
- Initialize a connection
- Get connection details
- Check Git status
- Get or create Git provider details

Merging Git branches



Git Logo by Jason Long is licensed under the [Creative Commons Attribution 3.0 Unported License](#).

Git-based deployments using workflows



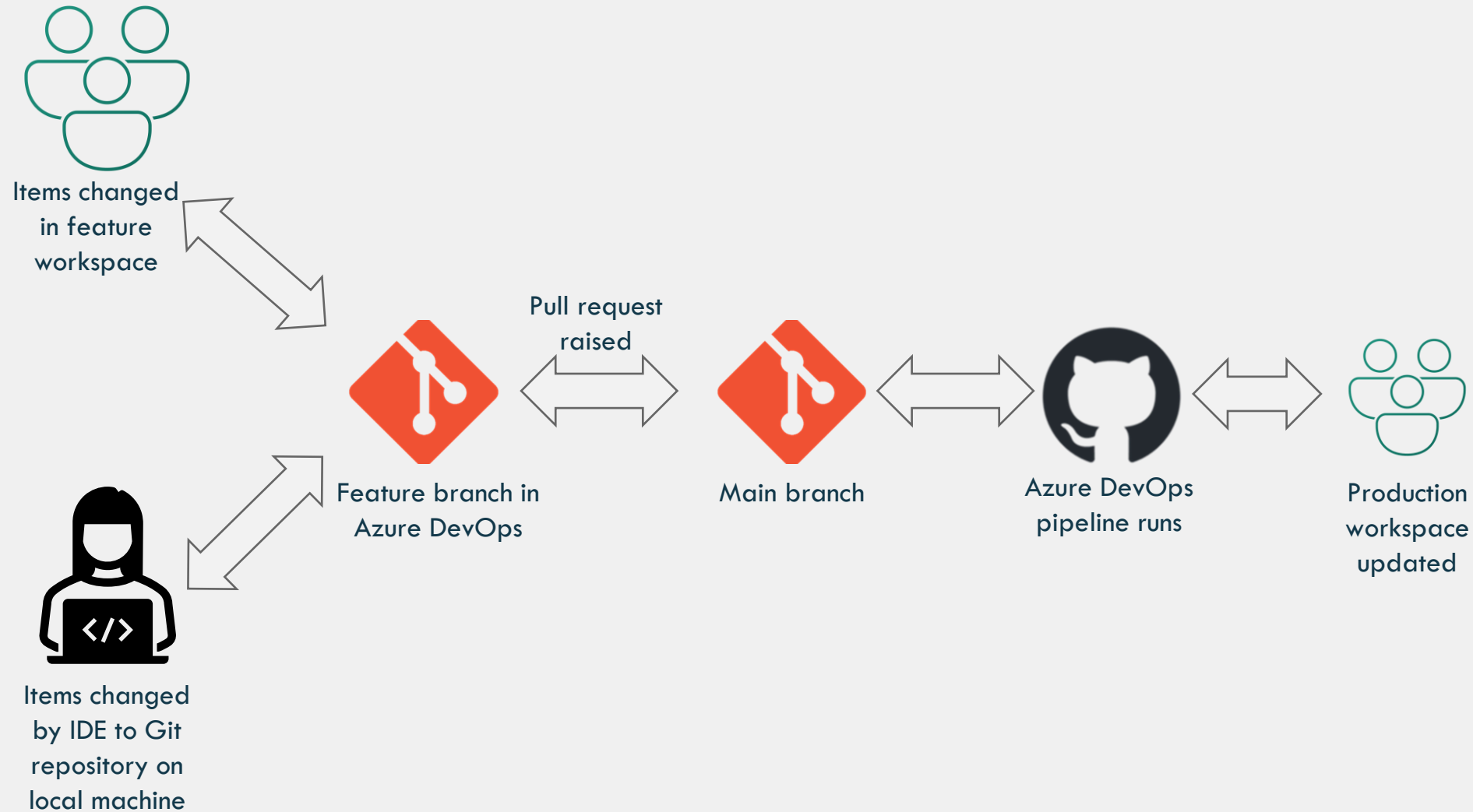
Workflows Terminology

- Workflow – Configurable process to run tasks
- Variables – predefined values (configured either in or out of pipeline)
- Secrets – Stores encrypted values natively in GitHub
- Parameters – Values entered when workflow started.
- Runner – Does all the work specified in the pipeline.

Deploy with Git-based sources

- Deploy to different workspaces using GitHub Actions workflows.
- Recommendation is that each CI/CD workflow contains a build and release process.
 - Build for unit tests.
 - Release to perform update.
- Use cases:
 - Require one branch to be source of truth.
 - Deploying to other tenants.

Deploy with Git-based sources

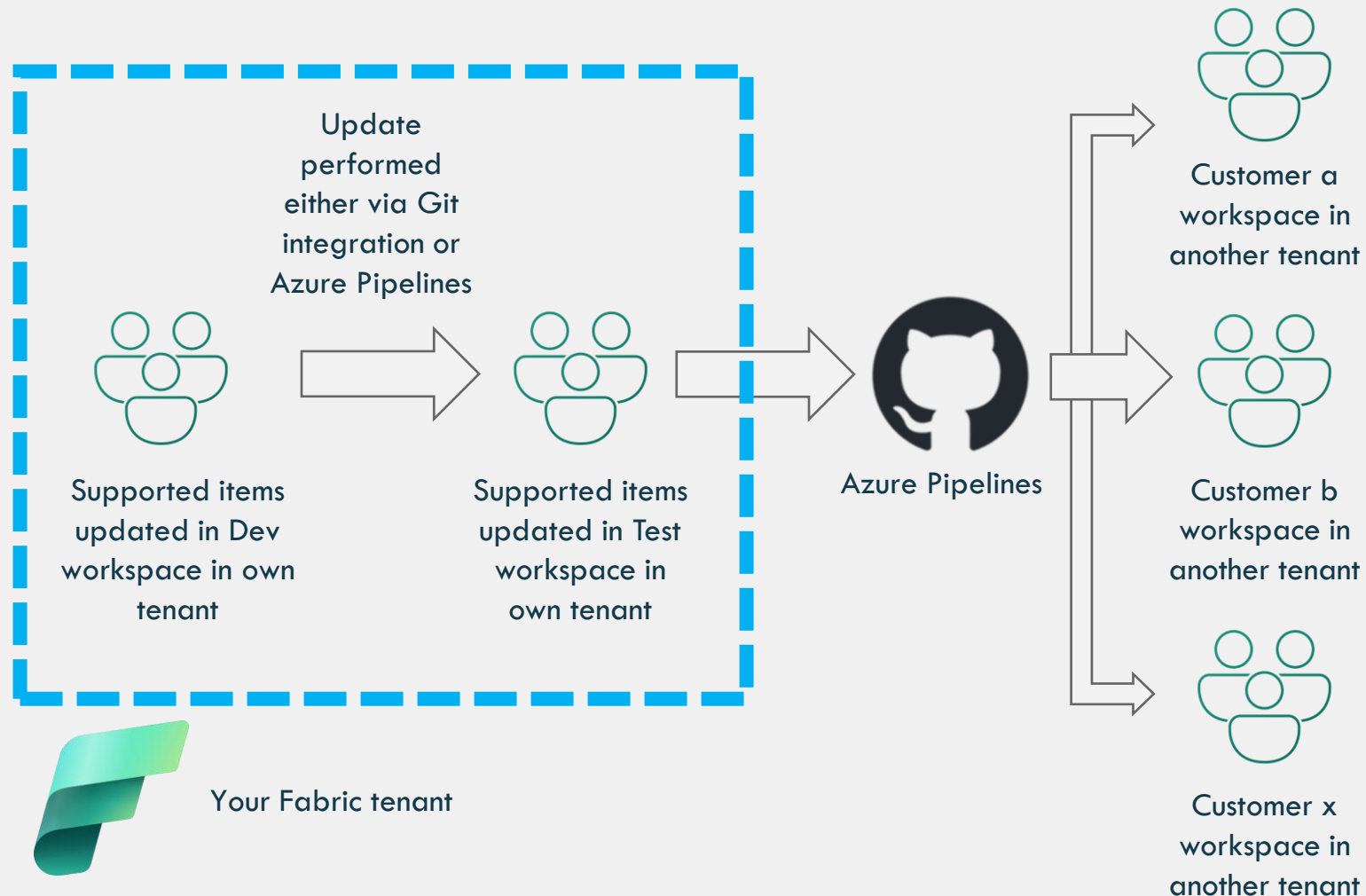


Git Logo by [Jason Long](#) is licensed under the [Creative Commons Attribution 3.0 Unported License](#).

For multiple customers/solutions/tenants

- Dev and test stages are managed in same Fabric tenant.
- Deployment to Prod stages to workspaces in other tenants using settings in workflow.
- Achieved by adding additional parameters or variables to a pipeline with supplied service principal details.
- Recommend using service principals and not user accounts.

Multi-tenant deployments



Build your DataOps support with CI/CD for Power BI and Fabric

Recommendations

- Avoid sensitive values in Pipelines. Store them in either:
 - GitHub encrypted secrets.
 - Azure Key Vault (preferred).
- Implement approvals process for production workloads.
- Suggest looking at tooling as layer of abstraction.

Additional tooling

Terraform

- Ideal when part of bigger IaC story
- Great for creating items

Fabric CLI

- Great for variety of tasks. Including CI/CD and governance.

PowerShell

- Custom modules available.
- Great for complex logic.

Fabric-cicd

- Deploys items stored with Git configuration elsewhere.
- Powerful for rebinding, esp with VL.

High-level overview of typical steps for fabric-cicd in a workflow

1. Point to repository configured with Git integration as source.
2. Specify Python version
3. Install fabric-cicd library.
4. Authenticate with Microsoft fabric tenant.
5. Run Python script.

Variable libraries

- Centralized configuration to store reusable values
- Supports environment-specific value sets.
- Works with Git integration.
- Works with various deployment methods, including deployment pipelines and fabric-cicd.

Pro tip: If you specify name of value sets before deployments and deploy with fabric-cicd stating aligned environment names the value sets will dynamically change accordingly.

Demos

- MicrosoftFabricMgmt PowerShell Module
- PowerShell & Fabric CLI deployments
- Deep dive into fabric-cicd deployments
- New fab deploy command

Exercises

- Fabric Exercise 2 – Local fabric–cicd deployment
- Fabric Exercise 3 – Fabric CLI deployment
- Fabric Exercise 4 – Fabric–cicd deployment

Advanced topics

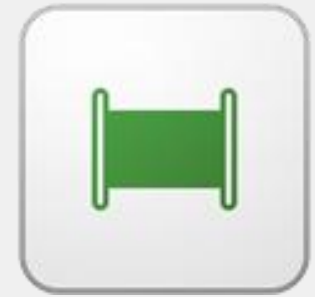
Items you can unit test include



Power BI items

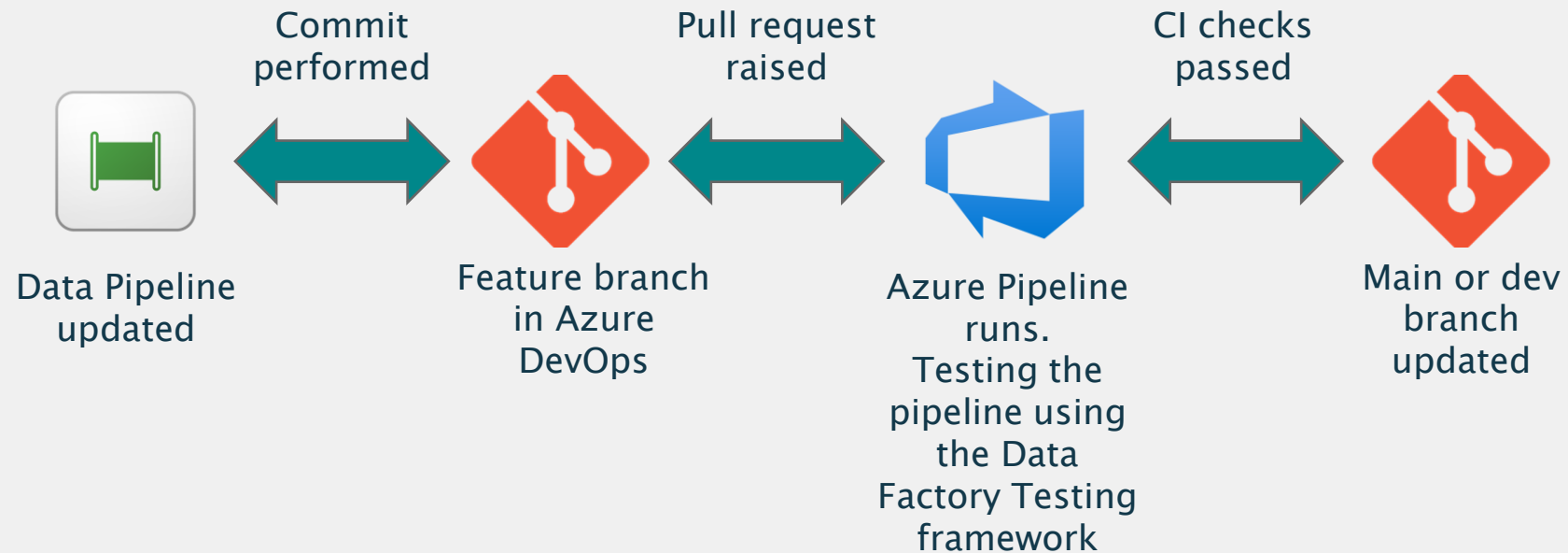


**Data
Warehouse**



Data Pipelines

Testing Data Pipelines



Git Logo by [Jason Long](#) is licensed under the [Creative Commons Attribution 3.0 Unported License](#).

Approvals

- Can be manual or automated.
- Manual approvals performed via environments.
- Environment approvals apply for certain accounts and all public repos.

Authentication

- Avoid storing sensitive information in a pipeline.
- Can use GitHub encrypted secrets natively in GitHub.
- Can also use Azure Key Vault or OpenID Connect (OIDC).

Creating your own GitHub Action

- Created via a GitHub repository created in a special way.
- You can build Docker container, JavaScript and composite actions.
- Can create for internal or public use.
- Can publish public ones on the GitHub Marketplace.
- Requires elements like tags and releases to be suitable for GitHub marketplace.

Enterprise security

- Authentication methods
- Managing secrets
- Effective governance for secrets

Advantages of GitHub Copilot

- Code creation
- Pull request checks
- Creating documentation

GitHub Copilot in VS Code

- Recommended essential for developers
- Various mode types
- Can be supercharged with MCP servers

Demos

- Approvals
- Anatomy of a GitHub Action
- Authentication
- GitHub Copilot
- GitHub Copilot in VS Code

Exercises

- Fabric Exercise 5 (optional) – Adding approval
- Fabric Exercise 6 (optional) – Create README file in existing GitHub Repo

Summarize CI/CD for Microsoft Fabric

- Git integration introduces new CI/CD possibilities.
- Multiple ways to perform CI/CD for Microsoft Fabric items.
- Secure deployments are achievable.
- Can be enhanced with AI.

Summarize day

- Power BI Projects introduce new AI possibilities
- GitHub can be used to store repositories
- GitHub Actions workflows can orchestrate deployments
- Enterprise grade features available

Questions & feedback



Build your DataOps support with CI/CD for Power BI and Fabric

Thank you



- Twitter/Bluesky: @kevchant
- LI: <https://www.linkedin.com/in/kevin-chant/>
- Blog: <https://www.KevinRChant.com>
- GitHub: <https://github.com/kevchant>

